Relational Database Systems (RDS)

Class no: 24

What is AWS RDS?

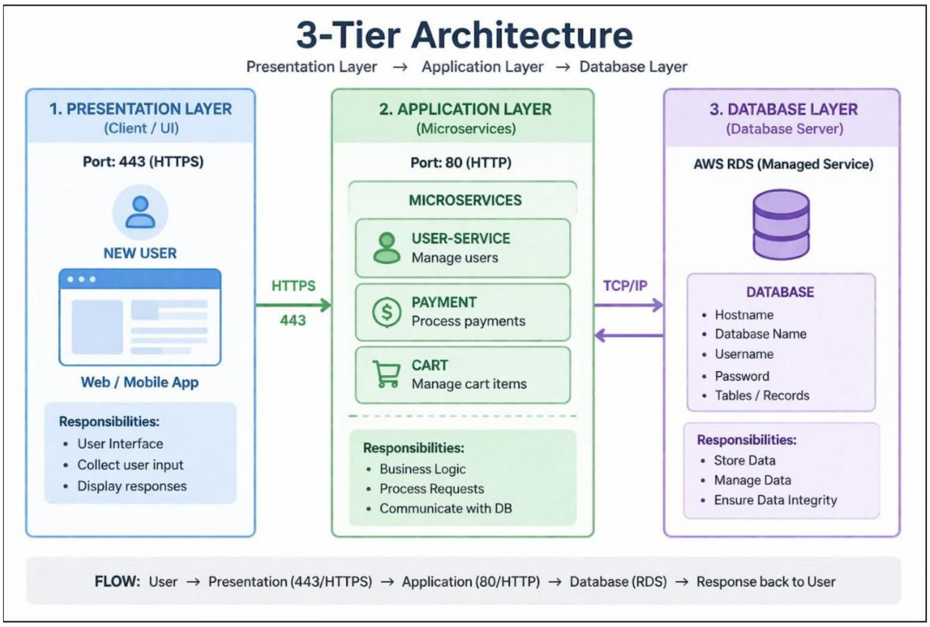
Amazon Relational Database Service (RDS) is a fully managed cloud service provided by AWS to simplify the setup, operation, and scaling of relational databases. It abstracts and automates many database administration tasks, such as:

- Hardware provisioning
- Database installation and configuration
- Patch management
- Automated backups and snapshots
- Monitoring and alerting
- Failover and disaster recovery

This allows developers and businesses to focus on application development rather than database maintenance, while A WS manages infrastructure and routine administrative tasks.

Key Advantages:

- Cost-efficient, resizable compute and storage capacities
- High availability via Multi-AZ deployments
- Automatic backups and point-in-time recovery
- Scalability through vertical scaling and read replicas
- Security using IAM, VPC isolation, SSL, and KMS encryption



Databases Supported by RDS

AWS RDS supports six major relational database engines, each offering distinct features:

1. **Amazon Aurora**
    - MySQL and PostgreSQL compatible, cloud-optimized, high performance
    - Automatically replicates data across multiple Availability Zones
2. **PostgreSQL**
    - Advanced open-source database
    - Supports complex queries, large datasets, GIS, and analytics applications
3. **MySQL**
    - Widely used open-source relational database
    - Well-suited for web applications, supports automated backups and Multi-AZ
4. **MariaDB**
    - Open-source fork of MySQL, compatible with existing MySQL applications
    - Offers additional storage engines and features for enhanced performance
5. **Oracle Database**
    - Enterprise-grade commercial database
    - Advanced SQL, PL/SQL, analytics, and robust transaction management
    - Licensing options: License Included (LI) or Bring Your Own License (BYOL)
6. **Microsoft SQL Server**
    - Popular enterprise database for Windows environments
    - Supports multiple editions: Express, Web, Standard, Enterprise
    - Features include TDE, replication, advanced reporting, and analytics

Note: Each engine comes with different management, parameter, and option groups that specify the behavior of the dat abase. Features may vary by engine, version, and region.

Default Port Numbers for RDS Databases

AWS RDS uses standard TCP ports for database connections, which can usually be customized if needed:

Database Engine	Default Port
PostgreSQL	5432

Database Engine	Default Port
MySQL / MariaDB	3306
Oracle Database	1521
Microsoft SQL Server	1433
Amazon Aurora	3306 (MySQL-compatible) or 5432 (PostgreSQL-compatible)

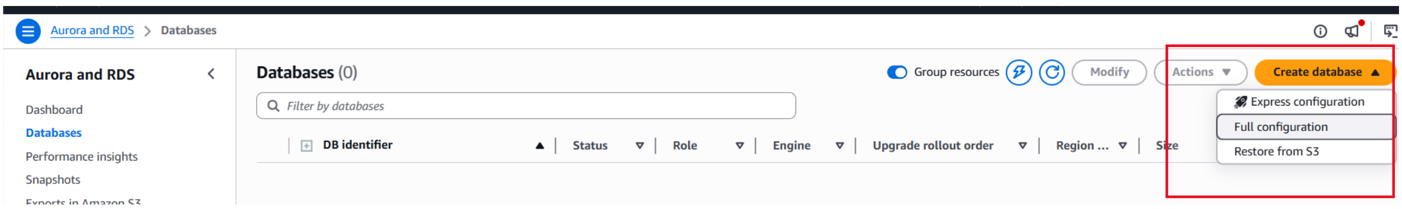
These ports must be opened in the RDS instance security group to allow client connections.

Hands on: Creating a RDS and connecting with an EC2 instance.

Step no:1 Search RDS → click **Aurora and RDS**.

Choose create Database → Full Configuration.

Express configuration is the database created automatically. Full configuration is where we choose and personalize the database.



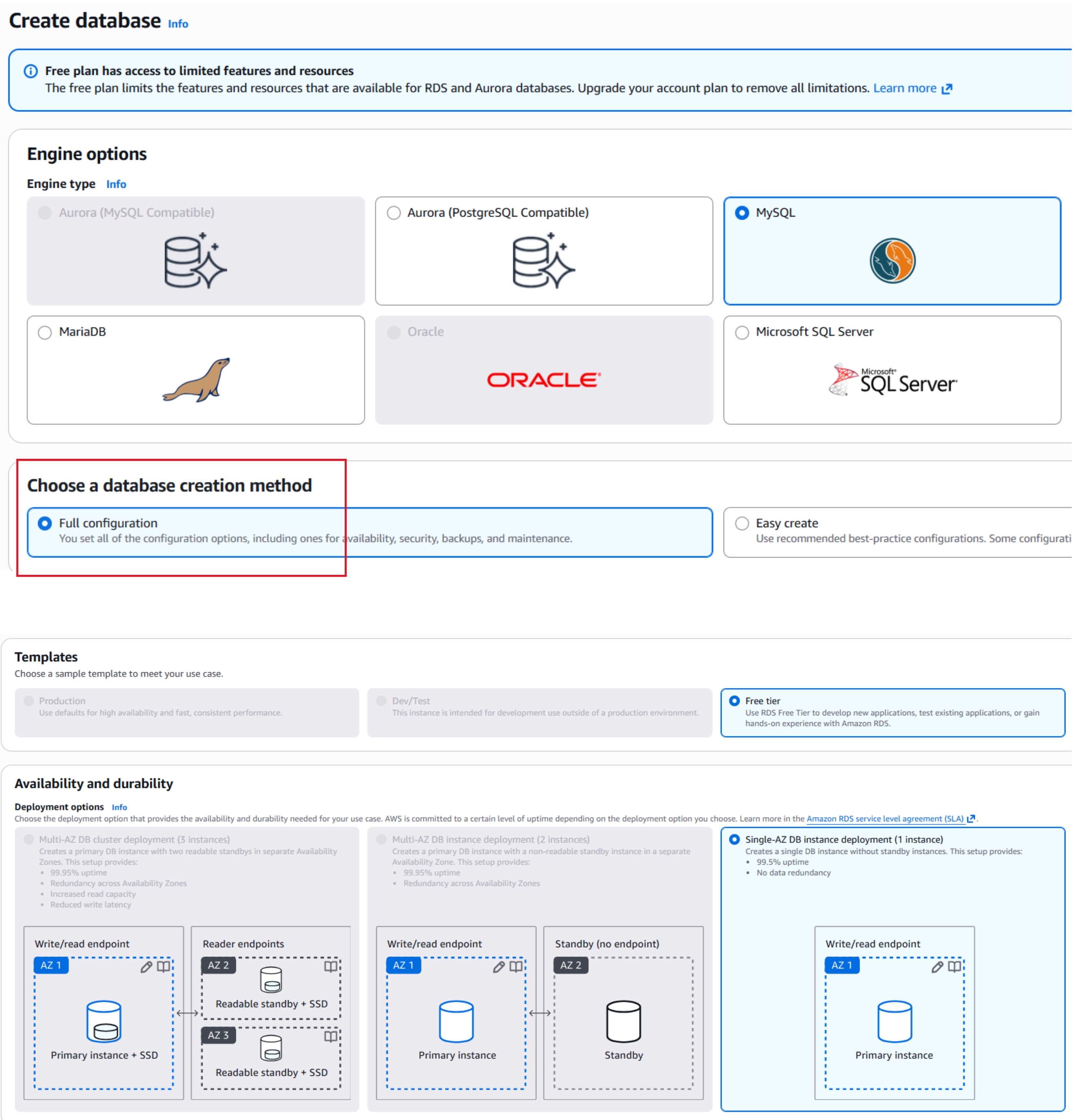
Step no:2 Choose **MySQL**,

Method: **Full configuration**

Template: Currently we are using free tier.

It normally depends on the environment we are creating the database like development, production.

Availability and durability: It depends on our need. More AZs more availability.

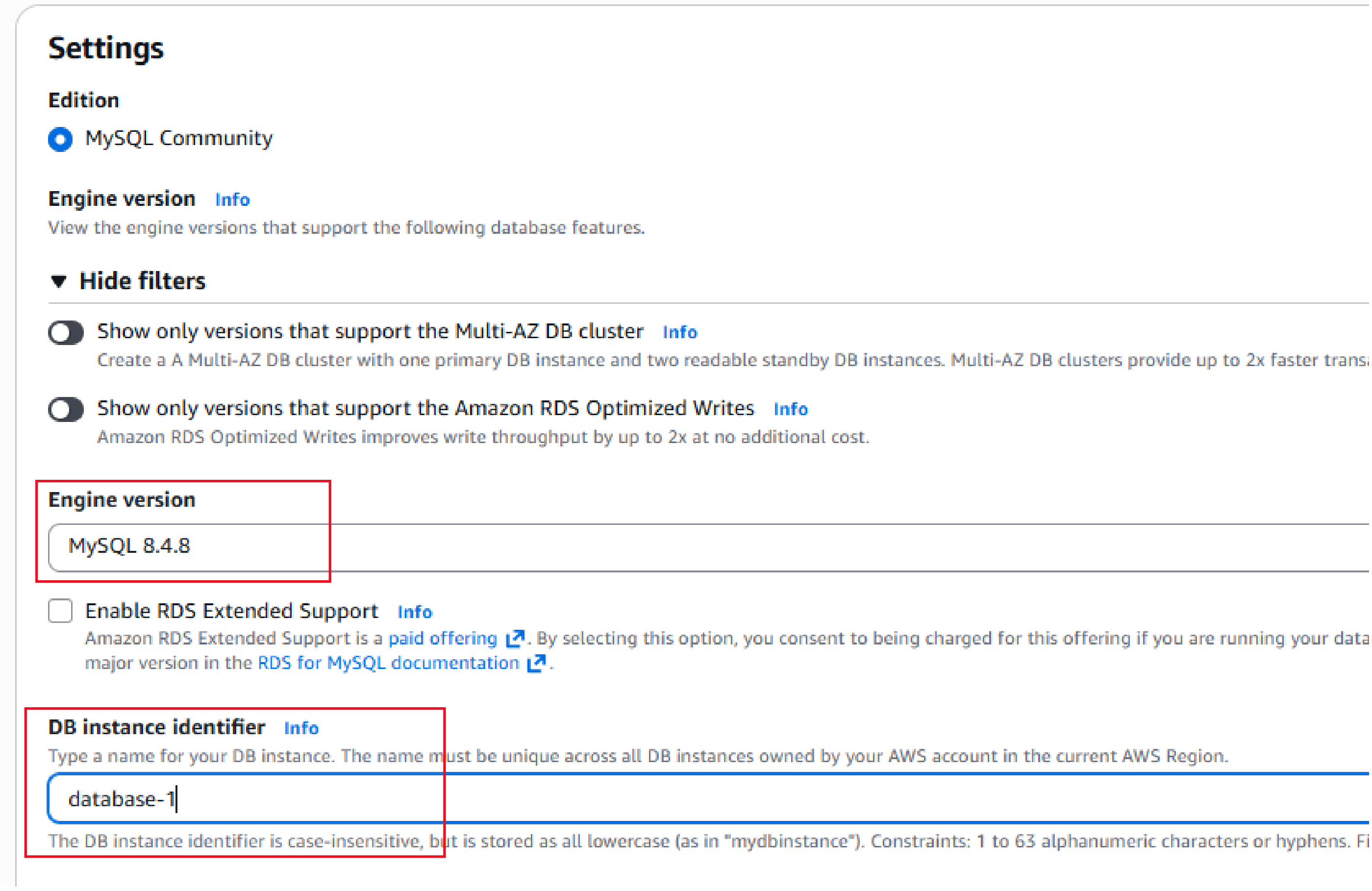


Step no:3 Enable → MySQL community.

Engine version: **Default (8.4.8)**

Disable: **RDS extension support**

DB- instance identifier: We provide the name of the name to identify. → Here it is **database-1.**



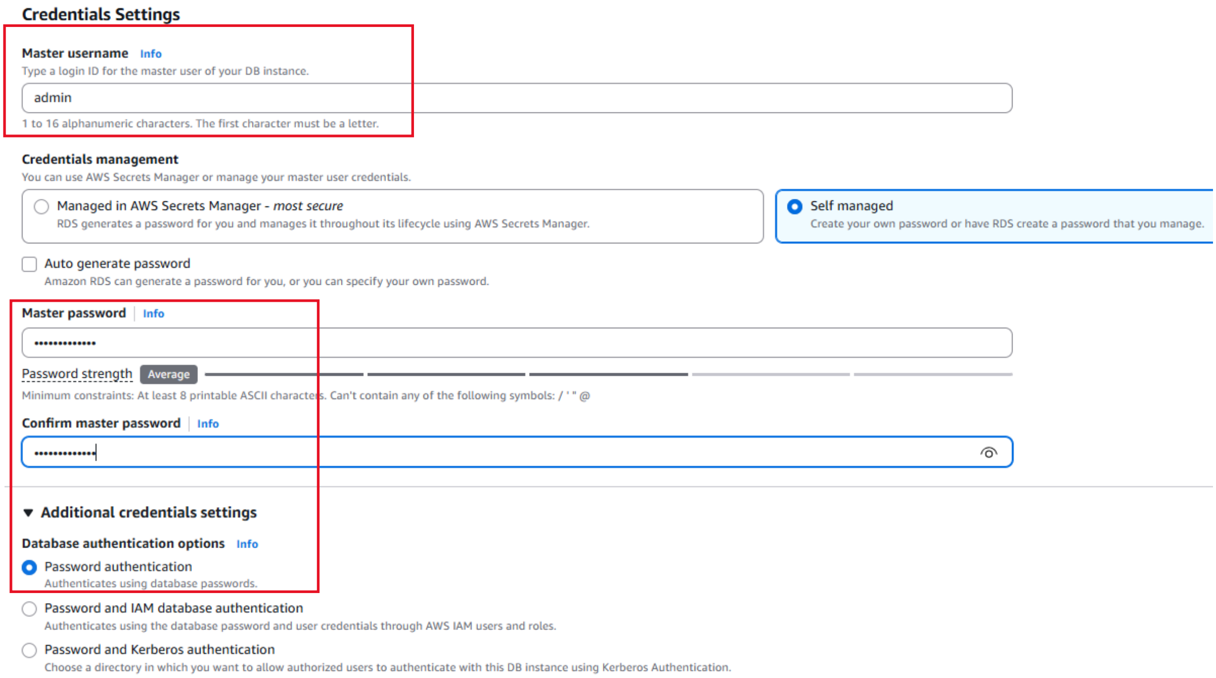
Credential settings,

Master username: **admin** (can perform READ and WRITE operations in database)

Password: **Self managed** → Create a password which does not contain any special characters.

Database authentication as password authentication.

(this username and password is used to connect with the microservices of the application)

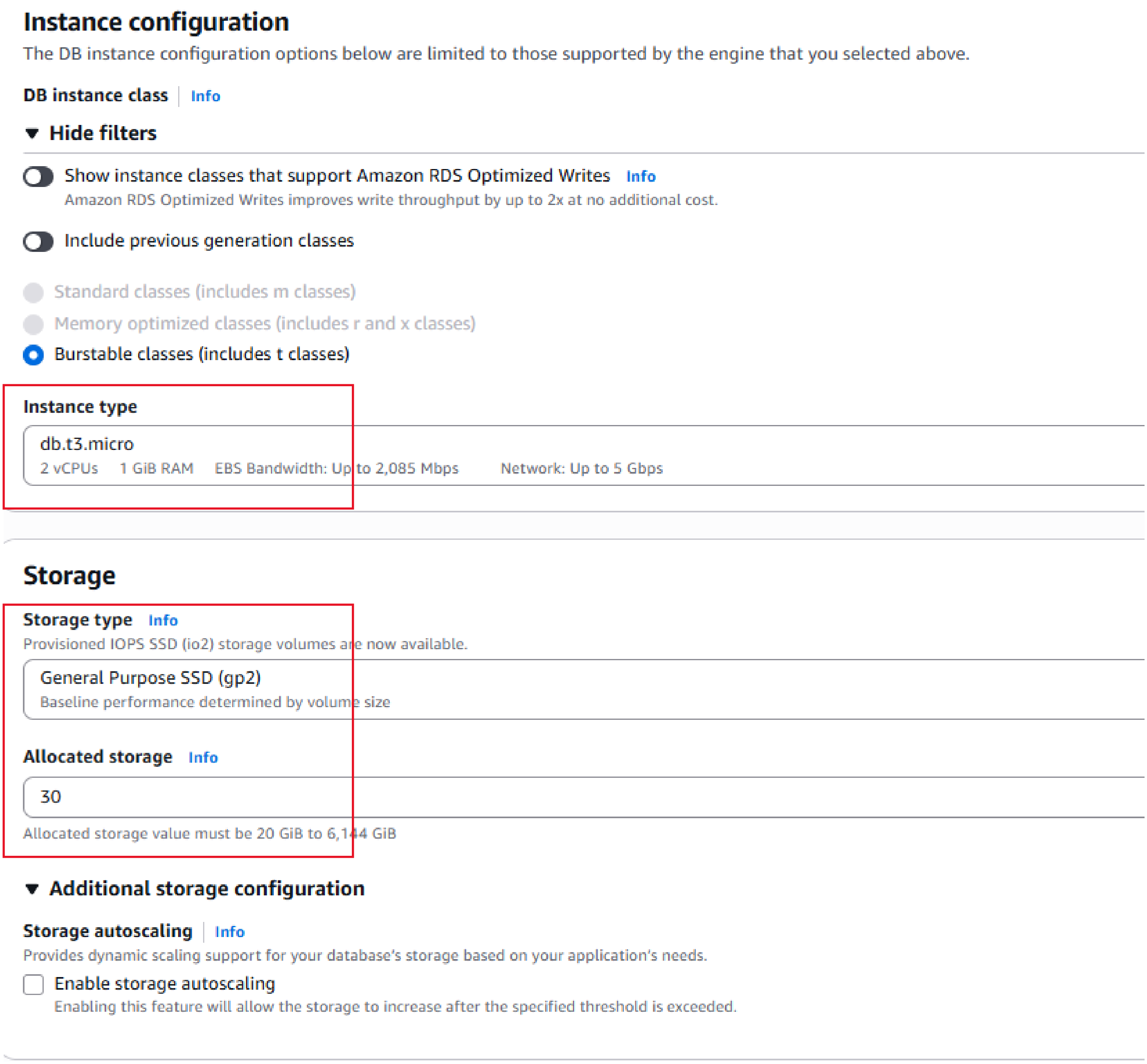


Instance configuration,

Instance type: **t3.micro** (As it is a database EC2 server)

Storage type: **General purpose SSD**

Allocated storage: **30**



Connectivity,

Compute resource: we will create an attach an EC2 later. Hence **Don't connect**.

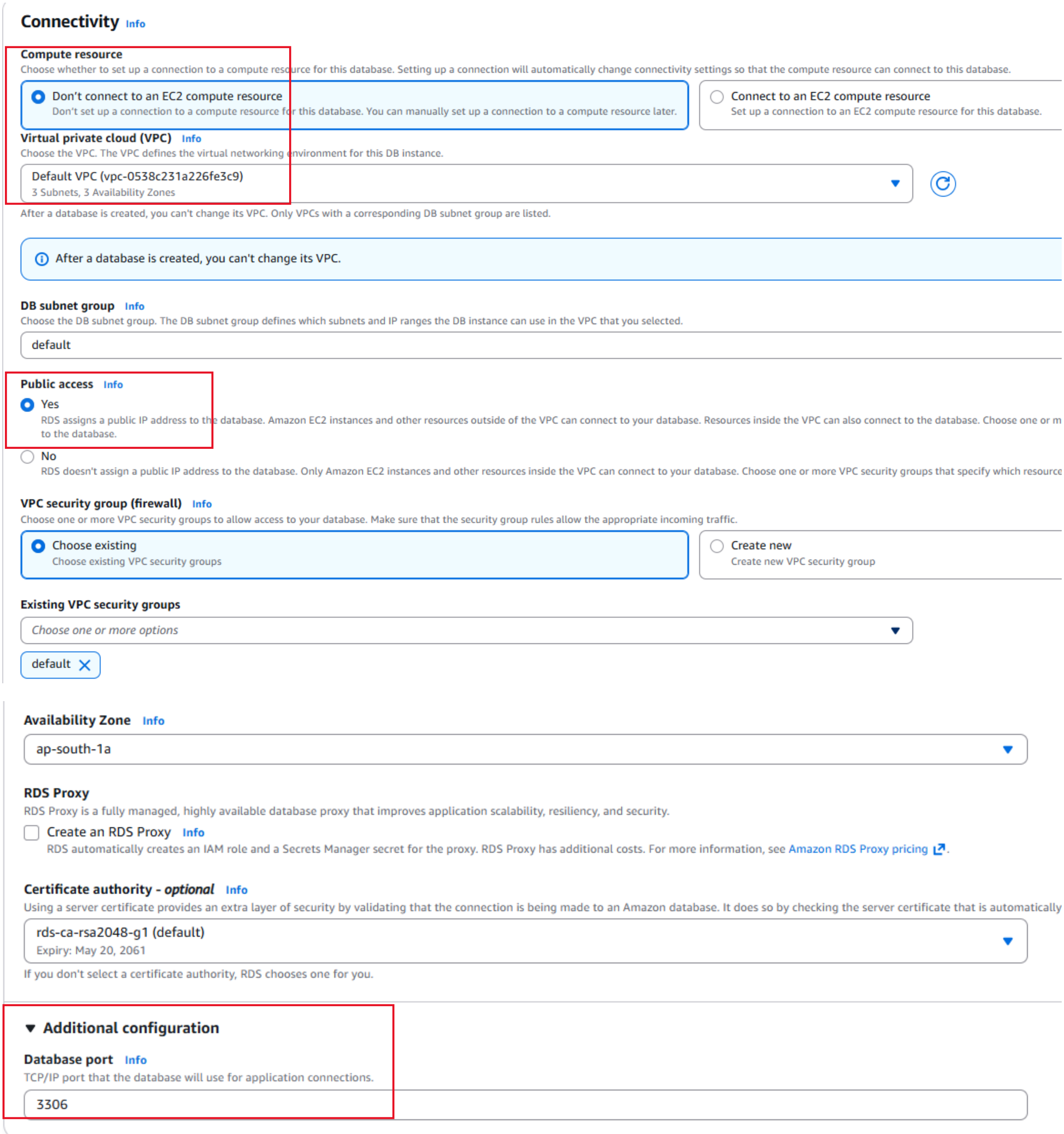
VPC: **Default VPC**

Subnet: **default**

Enable: **Public access**

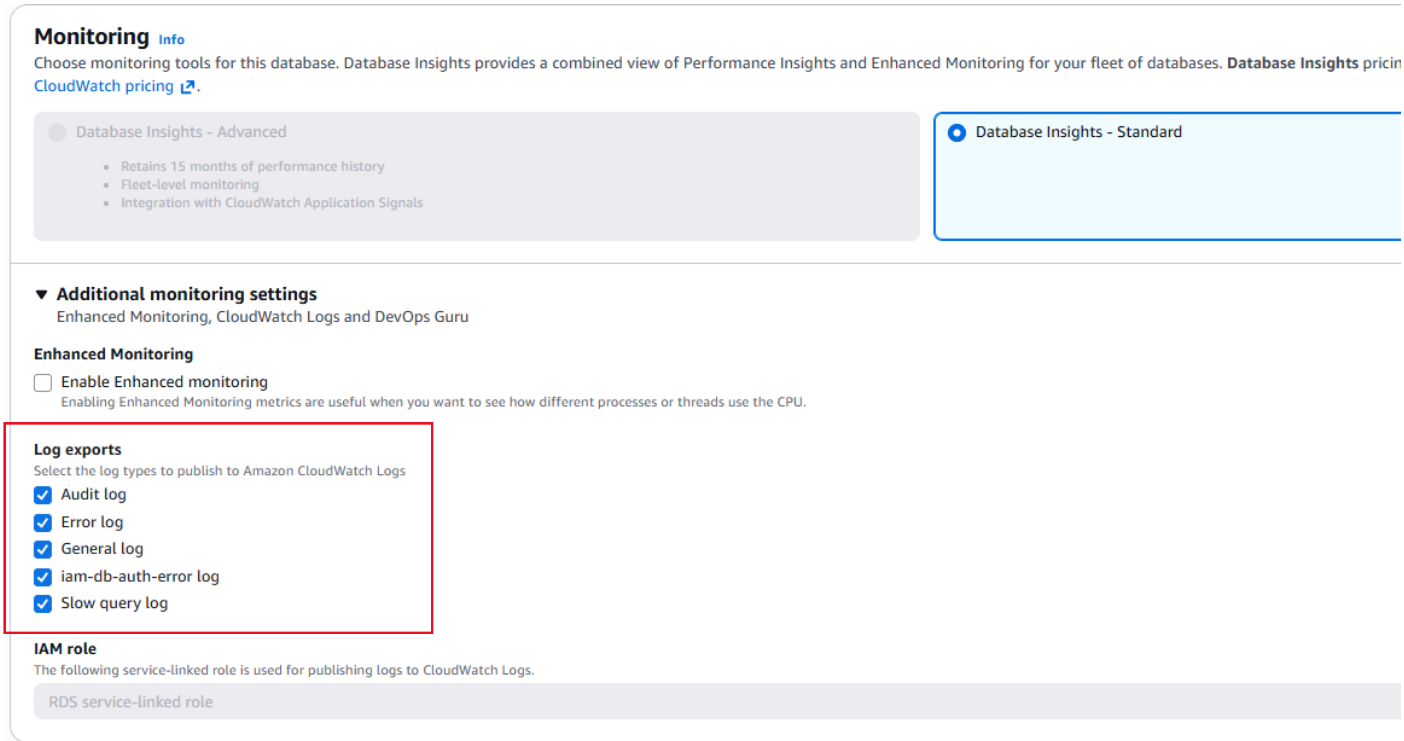
Security group: **Default** (ensure ALL TCP and MYSQL (3306) is added in inbound rules)

Choose the **AZs**



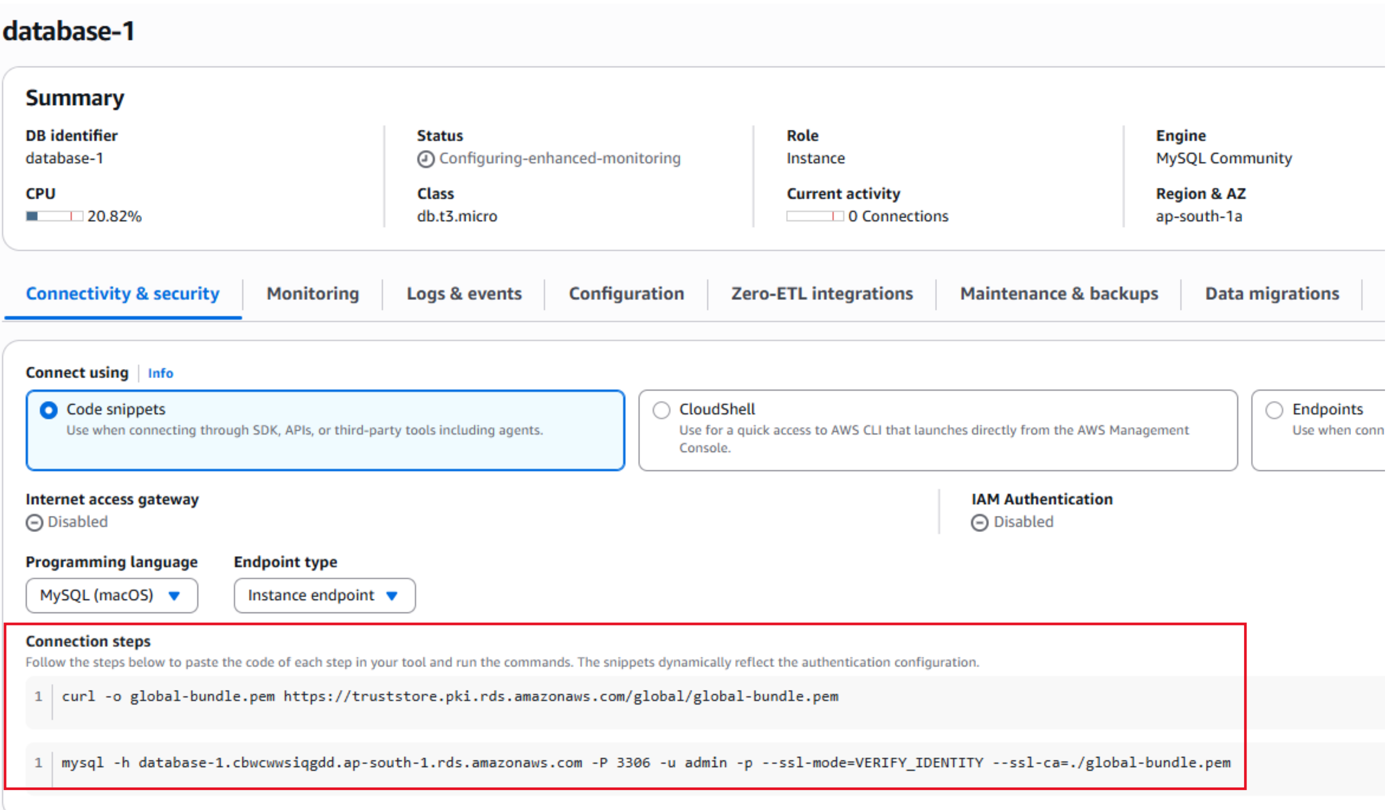
Monitoring,

We can choose log reports to monitor the database and use it for debugging purpose.



Click **Create database.**

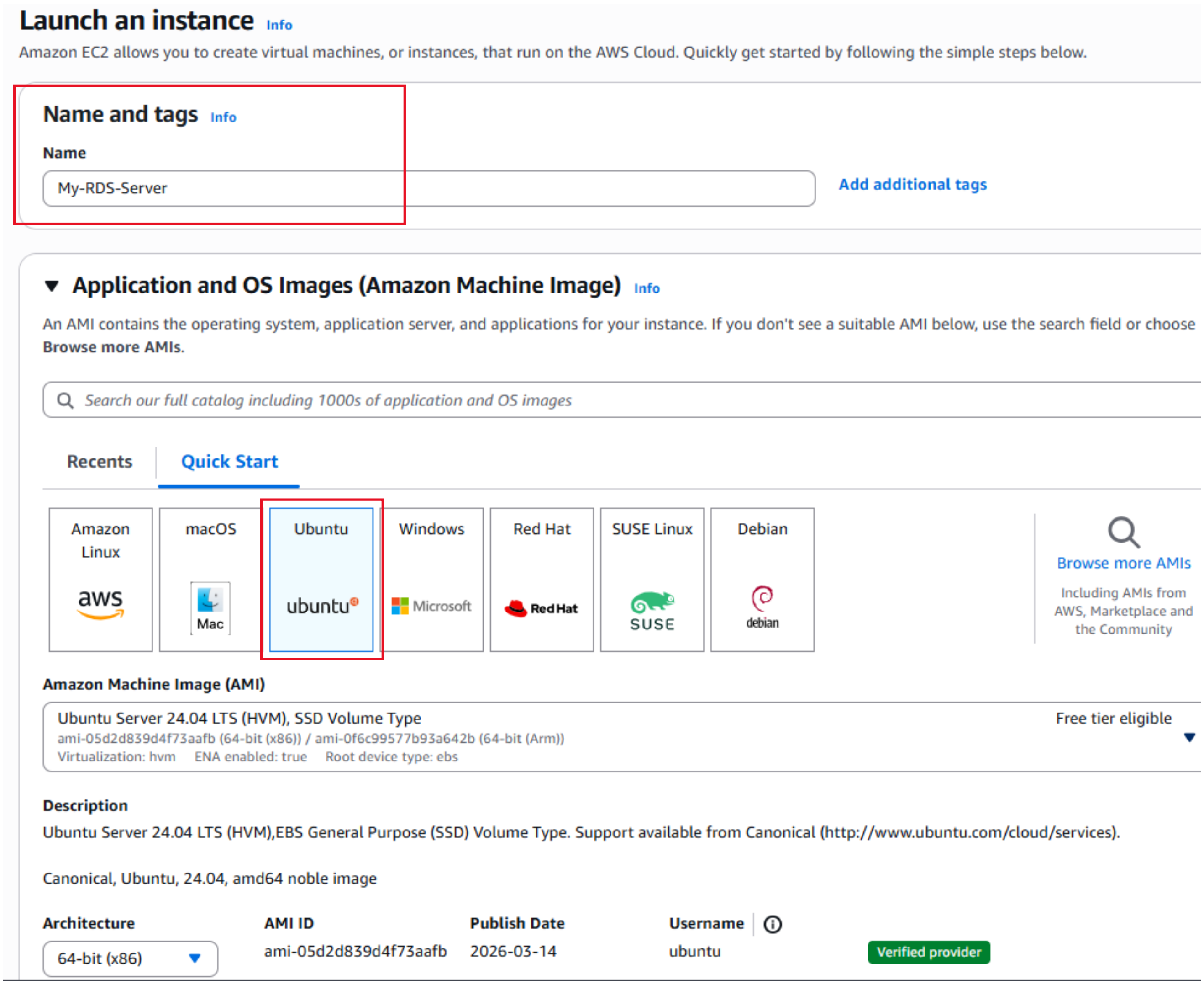
<u>**Step no:4**</u> Once created Click it, we can see the code to connect with an EC2 instance using the curl as well as mysql.



Step no:5 Create an Instance for connecting RDS Database.

Name: **My-RDS-Server**

OS type: **Ubuntu**



Instance type: **t3.micro**

Keypair: **Provide an available one or create a new one.**

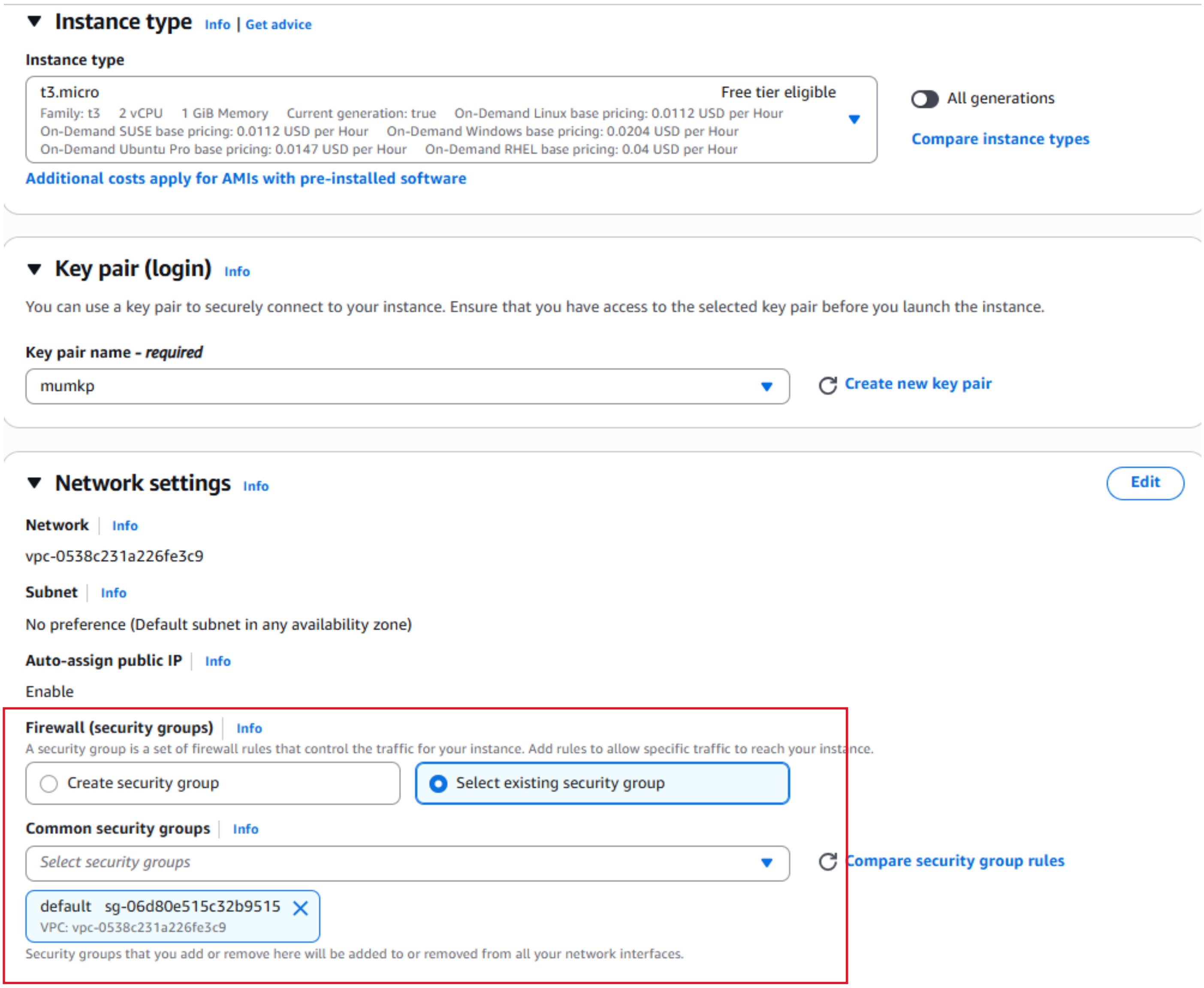
Network settings,

SG: **Default SG**

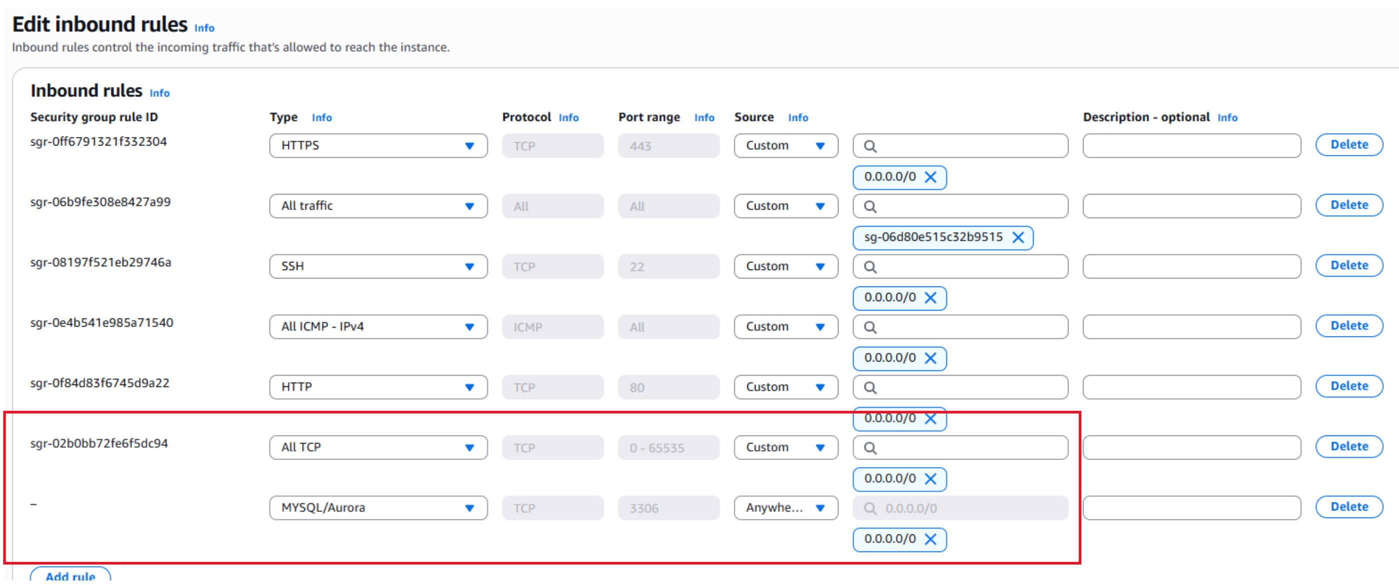
VPC**: Default VPC**

(ensure ALL TCP and MYSQL (3306) is added in inbound rules)

Launch inatance.



To add ALL TCP (0-65535) and for connection with MYSQL server enable it (default port number:3306) in the SG inbound rules.



Step no:6 Once the EC2 instance is created → select and click connect.

In order to connect with the RDS Of MYSQL type we need to install the MYSQL server.

Switch user: **sudo su**

Update the server: **apt update -y**

```
The list of available updates is more than a week old.
To check for new updates run: sudo apt update


The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>"
See "man sudo_root" for details.

ubuntu@ip-172-31-35-203:~$ sudo su
root@ip-172-31-35-203:/home/ubuntu# apt update -y
```

i-0ba4c2d1a6ea3ef0b (My-RDS-Server)

PublicIPs: 13.233.124.36 PrivateIPs: 172.31.35.203

Once updated, install MYSQL by: **apt install mysql-server**

```
root@ip-172-31-35-203:/home/ubuntu# apt install mysql-server
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  libcgi-fast-perl libcgi-pm-perl libclone-perl libencode-locale-perl libe
  libhttp-message-perl libio-html-perl liblwp-mediatypes-perl libmecab2 li
  mysql-server-8.0 mysql-server-core-8.0
Suggested packages:
  libdata-dump-perl libipc-sharedcache-perl libio-compress-brotli-perl lib
The following NEW packages will be installed:
  libcgi-fast-perl libcgi-pm-perl libclone-perl libencode-locale-perl libe
  libhttp-message-perl libio-html-perl liblwp-mediatypes-perl libmecab2 li
  mysql-server mysql-server-8.0 mysql-server-core-8.0
0 upgraded, 28 newly installed, 0 to remove and 44 not upgraded.
Need to get 29.6 MB of archives.
After this operation, 243 MB of additional disk space will be used.
Do you want to continue? [Y/n] Y
Get:1 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble/main amd64 mys
Get:2 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main a
```

Once installed,

Copy the **mysql command in the RDS database and paste it. (upto -p)**

It will ask for a password. Enter the **password we have created for the Master: admin.**

Now the instance will be connected with the RDS: **databse-1.**

Enter the database command: **SHOW DATABASES.** It is to see the available databases.

```
No VM guests are running outdated hypervisor (qemu) binaries on this host.
root@ip-172-31-35-203:/home/ubuntu# mysql -h database-1.cbwcwwsiqgdd.ap-south-1.rds.amazonaws.com -P 3306 -u admin -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 32
Server version: 8.4.8 Source distribution

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> SHOW DATABASES;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| sys                |
+--------------------+
4 rows in set (0.00 sec)
```

To create a database: **CREATE DATABSE database_name;**

To create a table: **CREATE TABLE table_name (Field_name, Type)**

```
mysql> CREATE DATABASE testdb;
Query OK, 1 row affected (0.03 sec)

mysql> USE testdb;
Database changed
mysql> CREATE TABLE Persons (
    ->   PersonID int PRIMARY KEY,
    ->   LastName varchar(255) NOT NULL,
    ->   FirstName varchar(255),
    ->   Address varchar(255),
    ->   City varchar(255)
    -> );
Query OK, 0 rows affected (0.06 sec)

mysql> SHOW DATABASES
    -> ^C
mysql> SHOW DATABASES;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| sys                |
| testdb             |
+--------------------+
5 rows in set (0.00 sec)

mysql>
```

i-0ba4c2d1a6ea3ef0b (My-RDS-Server)

PublicIPs: 13.233.124.36   PrivateIPs: 172.31.35.203

Normally, we will create a database with host, user name and password. Developer will use this in the micro services to connect with our database.

MASTER and READ Replicas

In Amazon RDS (Relational Database Service), the architecture of **Primary (Master)** and **Read Replicas** is designed to separate the "writing" of data from the "reading" of data. This improves performance and ensures your application stays snappy even under heavy traffic.

1. The Primary (Master) Instance

Think of the Master as the **"Source of Truth."** It is the main database instance that handles all the heavy lifting for data modifications.

- **Role:** Handles all **Write** operations (INSERT, UPDATE, DELETE) and **Read** operations.
- **Quantity:** There is only **one** Primary instance in a standard RDS setup.
- **Storage:** It holds the "Gold Copy" of your data.
- **Configuration:** Usually deployed in a **Multi-AZ** (Availability Zone) setup for high availability. If the Master fails, RDS automatically flips a switch to a standby instance.

2. Read Replicas

Read Replicas are "copies" of the Master instance used to offload work.

- **Role:** Handles **Read-only** queries (SELECT statements). You cannot write data directly to a replica.
- **Replication Type:** It uses **Asynchronous Replication**. This means there is a tiny delay (usually milliseconds) between data being written to the Master and appearing on the Replica.
- **Scaling:** You can create up to **15** replicas for most engines (like MySQL, PostgreSQL, and MariaDB) to handle massive amounts of read traffic.
- **Promotion:** If the Master dies and you don't have Multi-AZ, or if you just want to split the DB into two independent systems, you can **promote** a Read Replica to become its own standalone Master.

When should you use them?

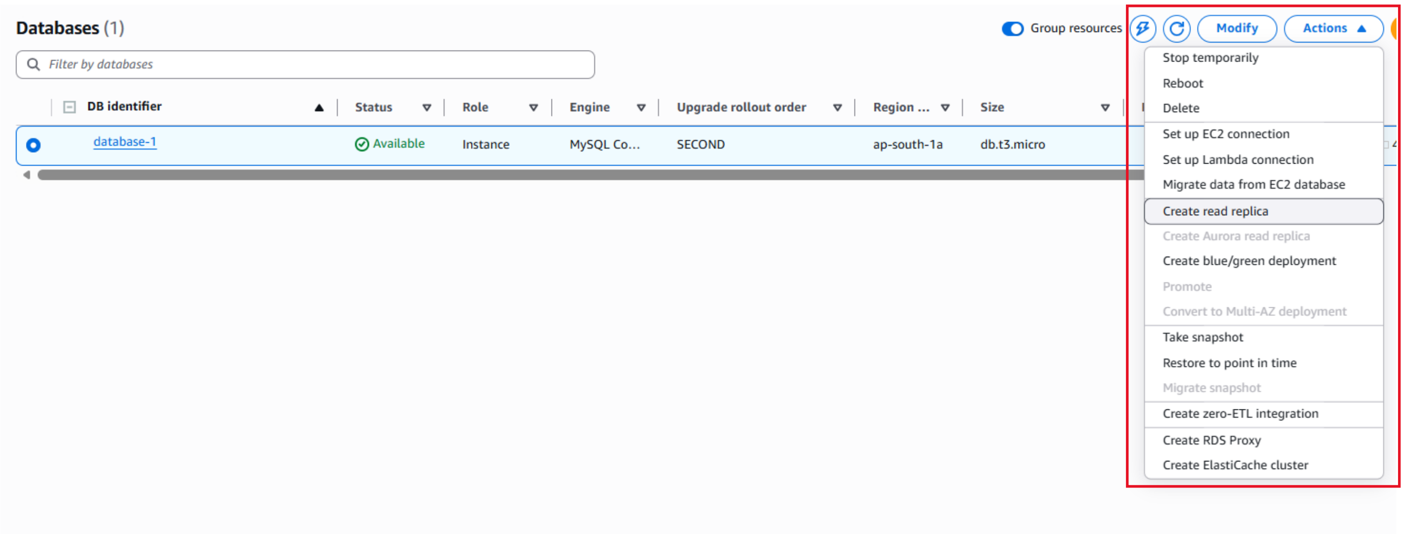
- **Heavy Reporting:** If your marketing team is running huge data reports that slow down the app, point their tools to a Read Replica.
- **Global Reach:** You can place Read Replicas in different **Regions** (e.g., a Master in US-East and a Replica in Tokyo) to reduce latency for international users.
- **Business Continuity:** While not a "backup" per se, they provide an extra layer of data redundancy.

Key Differences

Feature	Primary (Master)	Read Replica
Operations	Read & Write	Read-Only
Purpose	Data Integrity & Processing	Scalability & Performance
Replication	N/A	Asynchronous
Endpoint	Unique "Write" Endpoint	Unique "Read" Endpoint(s)
Billing	Standard RDS Rate	Billed as a separate instance

Hands on: Creating a read replica for an RDS database:

Step no: 1 Choose the created database → **actions** → **create read replica.**



Source: Our database named, **database-1**

DB instance identifier: Name of our read replica, **Mynewdb1**

Rest as default → Click **Create Read Replica.**

Create read replica

You are creating a replica DB instance from a source DB instance. This new DB instance will have the source DB instance's DB security groups and DB parameter groups.

Free plan has access to limited features and resources
The free plan limits the features and resources that are available for RDS and Aurora databases. Upgrade your account plan to remove all limitations. Learn more

Settings

Replica source
Source DB instance identifier

database-1
Role: Instance

DB instance identifier
This is the unique key that identifies a DB instance. This parameter is stored as a lowercase string (for example, mydbinstance).

Mynewdb1

Instance configuration

The DB instance configuration options below are limited to those supported by the engine that you selected above.

DB instance class | Info

Hide filters

Include previous generation classes

- Standard classes (includes m classes)
- Memory optimized classes (includes r and x classes)
- Burstable classes (includes t classes)

Instance type

db.t3.micro
2 vCPUs 1 GiB RAM EBS Bandwidth: Up to 2,085 Mbps Network: Up to 5 Gbps

Backup

Tags

- [ ] Copy tags to snapshots
- [x] Enable encryption
  Choose to encrypt the given instance. Master key IDs and aliases appear in the list after they have been created using the AWS Key Management Service console. Info

AWS KMS key Info

Enter a key ARN

Amazon Resource Name (ARN)

arn:aws:kms:ap-south-1:314383175192:key/65acb271-1648-4a80-ae32-10406ed9501b

Example: arn:aws:kms:<region>:<accountID>:key/<key-id>

Account
314383175192

KMS key ID
65acb271-1648-4a80-ae32-10406ed9501b

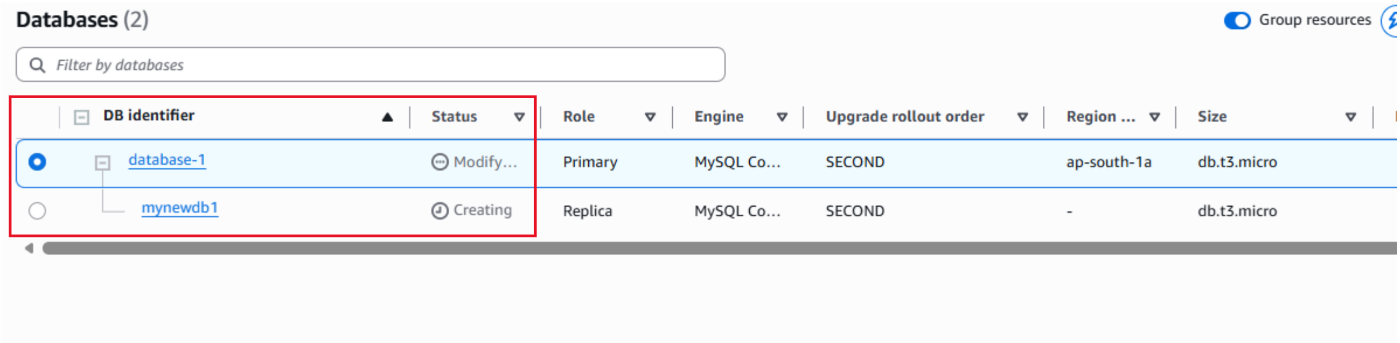
Maintenance

Auto minor version upgrade Info

- [x] Enable auto minor version upgrade
  Enabling auto minor version upgrade will automatically upgrade your database minor version. For limitations and more details, see Automatically upgrading the minor engine version documentation.
- [ ] Enable deletion protection
  Protects the database from being deleted accidentally. While this option is enabled, you can't delete the database.

Cancel | Create read replica

Once initiated we can see the replica under the respective database with status **creating.**



Once created, we can see the status as **available.**